Remember that there is still a server in serverless computing, but it is taken care of by the FaaS vendors like AWS, Microsoft, and Google.

For example, AWS offers a serverless computing environment in the name of Lambda functions. Developers can choose to build the applications as a set of Lambda functions that can be written in NodeJS, Java, Python, and a few other languages. AWS offers a ready-to-use environment to deploy these functions. It also offers ready-to-use database servers, file servers, application gateways, authentication servers, etc.

Similarly, Microsoft Azure offers an environment to build and deploy Azure functions in languages like C#.

Why serverless?

There are two main factors driving the popularity of serverless computing.

Ready-to-use environment: Obviously, this is the topmost selling point in favour of serverless computing. Businesses need not procure/book hardware or instances in advance, or worry about licences and setting up and provisioning the server. And they need not bother about scaling up and down. All of this is taken care of by the FaaS vendor.

Optimal cost: Since FaaS vendors always charge the clients based on the utilisation of the environment (pay as you use model), businesses need not worry about upfront costs and wastage of resources. For example, AWS charges the clients based on the number of requests a Lambda function receives, number of queries run on a table, etc.

Challenges with serverless computing

Like with any other approach, serverless computing is also not the perfect approach that everyone can blindly follow. It has its own set of limitations. Here are a few of them.

Vendor locking: The first and most important problem associated with serverless computing is that functions like Lambda or Azure are to be written using vendor-supplied APIs. For example, the functions that are written using an AWS Lambda API cannot be deployed into Google Cloud and vice versa. Hence, serverless computing forces businesses to commit to a vendor, for years. The success or failure of the application depends not only on the functionality but also on the ability of the vendor with respect to performance, etc.

Programming language: No serverless computing platform supports all the possible programming languages. Moreover, it may not support all the versions of a given programming language. The application development teams are constrained to choose only the languages that are offered. This may turn out to be very crucial in terms of the capabilities of the team.

Optimal cost, really?: It all depends on the usage of the resources. If your application is attracting a huge load, like millions of requests per second, the bill that you foot might be exorbitant. At such a scale, having your own server on-premises or on the cloud might work cheaper. It doesn't mean that applications with Web-scale are not suitable for serverless computing. It all boils down to the way you architect around the platform and the deal you sign with the vendor.

Ecosystem: No application is written for an isolated environment. It requires other components like data stores, databases, security engines, gateways, messaging servers, queues, cache, etc. Every platform offers its own set of such tools. For instance, AWS offers Dynamo DB as one of the NoSQL solutions. Obviously, other vendors offer their own NoSQL solutions. The teams are thus forced to architect the applications based on the chosen platform. Though most of the commercial FaaS vendors offer one or another component for a particular requirement, not every component may be best-in-class.

What about containers?

Many of us migrated to containerised deployment models in the last decade, since they offer a lightweight alternative to the costly machines, physical or virtual. With orchestration tools like Kubernetes, we love to deploy containerised applications and meet Web-scale requirements. Containers offer a certain degree of isolation from the underlying environment, which makes deployment relatively easy. However, we still need investments in hardware (on-premises or cloud), licences, networking, provisioning, etc, which demands forward planning, applicable technical skills, and careful monitoring. Serverless computing frees us even from these responsibilities, albeit with its own set of pros and cons.

Road ahead

We are in the days of continuous development, continuous integration, and continuous deployment. Every business is facing competition. Time to market (TTM) plays a significant role in gaining and retaining customers. In this backdrop, businesses love to spend more time churning out the functionality as quickly as possible rather than struggling with the nitty-gritty of deployment and maintenance. Serverless computing has the potential to meet these demands. Big players are committing huge investments to make FaaS as seamless and as affordable as possible. The future looks bright for serverless computing. END

By Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.